Project 1 report

CSE-490

Shouvik Das, Thomas Strade, Aditya Challamarad

Table of Contents:

Understanding of Project 1
Work Distribution 1
Description and Source Code 1
Thomas 2
Shouvik 3
Aditya 5
Aditya and Shouvik: 7
Discussion and Simulation 8
Schematics 9
Datapath 9
Control Unit 9
References 9
Conclusion 9

Understanding of Project

The goal of this project was to design, implement, and simulate a simple 16-bit single-cycle processor using Verilog HDL. The processor follows MIPS-like architecture and supports three instruction formats: R-type, I-type, and J-type. A total of 11 instructions were implemented, including arithmetic and logic operations (add, sub, addi, sll), memory access (lw, sw), branch instructions (beq, bne), and unconditional jump (jmp). The processor was built using individual components, including the Program Counter, Instruction Memory, Register File, ALU, Data Memory, Control Unit, and supporting modules (Mux, Sign Extender). Each of these components was built using behavioral Verilog and then combined at the top using structural Verilog. The completed design was simulated using Xilinx Vivado and deployed onto a Xilinx Basys 3 FPGA board for hardware verification.

Work Distribution

Thomas

- ALU
- Register File
- Data Memory

- Branch / Jump

Shouvik

- PC
- Adder
- Sign Extender
- Control Unit

Aditya

- Instruction Memory
- Multiplexer
- Shifter
- Control Unit

Description and Source Code

Thomas:

Arithmetic Logical Unit (ALU)

Inputs:

- A, B: 16-bit operands
- Operation: 3-bit control signal

Outputs:

- Result: 16-bit operation result
- Flags: Equal / Not equal flags

The ALU runs the input control signals through a case statement to apply the correct operation to the operands. The inputs are then checked for equality and set the flags accordingly.

```
module ALU(
    input wire[15:0] A,
    input wire[15:0] B,
    input wire[2:0] Operation,
    output wire[15:0] Result,
    output wire[1:0] Flags
    );

    reg[15:0] r;
    reg[1:0] f;

    localparam EQ = 1;
    localparam NE = 0;

    always@ (A, B, Operation)
```

```
    begin
        case(Operation)
            3'b000: r = A + B;
            3'b001: r = A - B;
            3'b010: r = B << A;
            3'b011: r = A & B;
            3'b100: r = A >> B;
            3'b101: r = A | B;
            3'b110: r = A ^ B;
            3'b111: r = ~A;
        endcase

        if (A == B) begin
            f[EQ] = 1'b1;
            f[NE] = 1'b0;
        end
        else if (A != B) begin
            f[EQ] = 1'b0;
            f[NE] = 1'b1;
        end
        else begin
            f[EQ] = 1'b0;
            f[NE] = 1'b0;
        end
    end

    assign Result = r;
    assign Flags = f;

endmodule
```

Register File

Inputs:

- ReadReg1, ReadReg2: 4-bit register sources
- WriteData: 16-bit control data to be written
- Ctrl_RegWrite: 1-bit control signal for writing to a register
- Clock: System clock for synchronization

Outputs:

- ReadData1, ReadData2: 16-bit data from 2 input registers

The Register file is the primary module that is responsible for reading from the register, transferring its contents, and also writing to a destination register.

```
module RegisterFile(
    input wire[3:0] ReadReg1,
    input wire[3:0] ReadReg2,
    input wire[15:0] WriteData,
    input wire Cntrl_RegWrite,
```

```
    input wire Clock,
    output wire[15:0] RegData1,
    output wire[15:0] RegData2
    );

    reg [15:0] regfile [0:15];
    integer i;

    reg [15:0] regdata1;
    reg [15:0] regdata2;

    initial
    begin
        for (i = 0; i < 16; i = i + 1) begin
            regfile[i] = 16'h0000;
        end
    end

    always@ (Clock)
    begin
        if (Clock) begin
            regdata1 = regfile[ReadReg1];
            regdata2 = regfile[ReadReg2];
        end
        else if (!Clock & Cntrl_RegWrite) begin
            regfile[ReadReg1] = WriteData;
        end
        else ;
    end

    assign RegData1 = regdata1;
    assign RegData2 = regdata2;

endmodule
```

Data Memory

Inputs:

- Addr: 16-bit operand which is the address in memory.
- DataIn: 16-bit data that is to be written into memory.
- MemRead/MemWrite: 1-bit control signal for Reading/writing to memory

Outputs:

- DataOut: 16-bit data that is retrieved from memory and written to a register

The Data Memory is the system primary memory. It is addressed by 16 bit values and is responsible for storing any data that the processor needs (after it has been written to).

```
Module DataMemory(
    input wire[15:0] Addr,
    input wire[15:0] DataIn,
```

```
    input wire MemRead,
    input wire MemWrite,
    output wire[15:0] DataOut
    );

    reg[7:0] datamem [0:255];
    integer i;

    reg [7:0] a;
    reg [15:0] dout;

    initial
    begin
        for (i = 0; i < 256; i = i + 1) begin
            datamem[i] = 8'h00;
        end
        dout = 16'h0000;
    end

    always@ (Addr, DataIn, MemRead, MemWrite)
    begin
        a = Addr[7:0];

        if (MemRead == 1) begin
            dout = { datamem[a], datamem[a + 1] };
        end
        else if (MemWrite == 1) begin
            datamem[a] = DataIn[15:8];
            datamem[a + 1] = DataIn[7:0];
        end
        else ;
    end

    assign DataOut = dout;
endmodule
```

Branch Logic

Inputs:

- BranchCntrl: 2-bit control wires for selecting Branch EQ/Not EQ
- ALUFlags: 2-bit ALU flags for determining if branch should take place

Outputs:

- Result: 1-bit control wire for signaling to branch or not

Decides if condition like BEQ or BNE has been met. IF the ALU flags match the branch requested, it tells processor to skip to the branch address, otherwise, it just moves to next instruction

```
Module BranchLogic(
    input wire[1:0] BranchCntrl,
```

```
    input wire[1:0] ALUFlags,
    output wire Result
    );

    reg r;

    always@ (BranchCntrl, ALUFlags)
    begin
        if ((BranchCntrl == 2'b01) && (ALUFlags == 2'b01))
            r = 1'b1;
        else if ((BranchCntrl == 2'b10) && (ALUFlags == 2'b10))
            r = 1'b1;
        else
            r = 1'b0;
    end

    assign Result = r;

endmodule
```

Jump Logic

Inputs:

- PC: Current 16-bit Program Counter address.
- Offset: 16-bit immediate value from the instruction.

Outputs:

- Result: The 16-bit target address for the jump.

The Jump Logic module performs addition between the current PC and the instruction offset. This calculates the memory address the processor should jump to if the Jump signal is given.

```
Module JumpLogic(
    input [15:0] PC,
    input [15:0] Offset,
    output [15:0] Result
    );

    reg [15:0] jmp_addr;

    always@ (PC, Offset)
    begin
        jmp_addr = PC + Offset;
    end

    assign Result = jmp_addr;

endmodule
```

Shouvik:

Program Counter (PC)

Inputs:

- NextAddr: 16-bit Address of Next Instruction

Outputs:

- CurrAddr: 16-bit Address of Current Instruction

The PC works on assigning the address of the NEXT instruction which is the input to the program counter, as the address of the CURRENT instruction and outputs that, so that the program can correctly run the instructions one after the other.

```
module PC(
    input wire[15:0] NextAddr,
    input wire Clock,
    output wire[15:0] CurrAddr
    );

    reg[15:0] pc_reg;
    initial

    begin
        pc_reg = 16'h0000;
    end

    always@ (posedge Clock)
    begin
        pc_reg <= NextAddr;
    end

    assign CurrAddr = pc_reg;

endmodule
```

Adder

Inputs:

- A and B: 16-bit values each

Outputs:

- Result: 16-bit Result

The Adder works on adding two 16-bit input values and returning a 16-bit result.

```
module Adder(
    input [15:0] A,
    input [15:0] B,
    output [15:0] Result
    );
```

```
    reg [15:0] r;

    always@ (A, B)
    begin
        r = A + B;
    end

    assign Result = r;

endmodule
```

Sign Extender

Inputs:

- ImmediateIn: 4-bit immediate input

Outputs:

- ImmediateOut: 16-bit immediate sign-extended value

The SignExtender takes in a 4-bit immediate input, takes the MSB, which in this case is bit 3, and copies that to all the remaining bits [15:4], thus outputting a 16-bit immediate value that is signed extended.

```
module SignExtender(
    input wire[3:0] ImmediateIn,
    output wire[15:0] ImmediateOut
    );

    reg [15:0] extended;

    always@ (ImmediateIn)
    begin
        extended = {{12{ImmediateIn[3]}}, ImmediateIn[3:0]};
    end

    assign ImmediateOut = extended;

endmodule
```

Aditya:

Multiplexer 2-to-1 (Multiplexer)

Inputs:

- A,B: Two 16-bit operands
- Select: Wire to Select input

Output

- Out: 16-bit result

The 2-to-1 multiplexer takes two 16-bit data inputs, and a 1 bit select bit and is used to decide which input is chosen and passed to the output. This multiplexer will be used throughout the design when we need to choose between 2 inputs.

```
module MUX2to1(
    input wire[15:0] A,
    input wire[15:0] B,
    input wire Select,
    output wire[15:0] Out
    );

    reg[15:0] data_out;

    always@(A,B,Select)
    begin
        case(Select)
            1'b0: data_out = A;
            1'b1: data_out = B;
        endcase
    end

    assign Out = data_out;
endmodule
```

Left Shifter

Inputs:

- Data_In: 16-bit input

Outputs:

- Data_Out: 16-bit left shifted output

The left shifter takes in a 16-bit input and multiplies the input by 2, shifting it left by 1 (<< 1), and returning it in Data_Out.

```
module LeftShifter(
    input [15:0] DataIn,
    output [15:0] DataOut
    );

    reg [15:0] tmp = 16'h0000;

    always@ (DataIn)
    begin
```

```
        tmp = DataIn << 1;
    end

    assign DataOut = tmp;

endmodule
```

Instruction Memory

Inputs:

- Addr: 16-bit address input

Outputs:

- Opcode: 4-bit instruction opcode
- Rs: 4-bit source register identifier
- Rt: 4-bit target register identifier
- Funct: 4-bit function code
- Immed: 4-bit immediate value
- JmpOffset: 12-bit jump offset

The instruction memory takes a 16-bit address and reads the 16-bit instructions from internal memory, then it splits this instruction into the Opcode, Rs, Rt, Funct, Immed, and JmpOffset fields to be used by the rest of the datapath.

```
module InstructionMemory(
    input wire[15:0] Addr,
    output wire[3:0] Opcode,
    output wire[3:0] Rs,
    output wire[3:0] Rt,
    output wire[3:0] Funct,
    output wire[3:0] Immed,
    output wire[11:0] JmpOffset
    );

    reg [15:0] instrmem [0:127];
    reg [15:0] instruction;

    initial
    begin
        $readmemh("instructions_memoryfile.mem", instrmem, 16'h0000, 16'h007F);
    end

    always@ (Addr)
    begin
        instruction = instrmem[Addr[6:0]];
    end
```

```
    assign Opcode = instruction[15:12];
    assign Rt = instruction[11:8];
    assign Rs = instruction[7:4];
    assign Funct = instruction[3:0];
    assign Immed = instruction[3:0];
    assign JmpOffset = instruction[11:0];
endmodule
```

Aditya and Shouvik:

Control Unit

Inputs:

- Opcode: 4-bit opcode from an instruction

Outputs:

- Branch: 2-bit control wire for branch equal/not equal.
- Jump: 1-bit control wire for Jump instructions
- MemRead: 1-bit control wire for controlling if the processor should read from memory
- MemWrite: 1-bit control wire for controlling if the processor should write to memory
- ALUSrc: 1-bit control wire for selecting if we should pass Source reg 2 into ALU.
- MemToReg: 1-bit control wire for setting if data from memory should be written into a register.
- RegWrite: 1-bit control wire for controlling if data should be written into a register.
- ALUOp: 2-bit control wire for selecting what operation the ALU should perform.

The Control unit takes in a 4-bit opcode, creates a lookup table (LUT) to map the control signals depending on each valid opcode, then the word is split into multiple different control signals that will control the Datapath's behavior, memory access, ALU operations, register operations, and branching based on the current instructions.

```
module ControlUnit(
    input wire[3:0] Opcode,
    output wire[1:0] Branch,
    output wire Jump,
    output wire MemRead,
    output wire MemWrite,
    output wire ALUSrc,
    output wire MemtoReg,
    output wire RegWrite,
    output wire[1:0] ALUOp
    );

    reg [9:0] csLUT [0:15];
    reg [9:0] cntrl_sig;
    integer i;
```

```
    initial
    begin
        csLUT[0] = 10'b0000000101; // add, sub, sll, and
        csLUT[1] = 10'b0001011100; // lw
        csLUT[2] = 10'b0000110000; // sw
        csLUT[3] = 10'b0000010100; // addi
        csLUT[4] = 10'b1000000001; // beq
        csLUT[5] = 10'b0100000001; // bne
        csLUT[6] = 10'b0010000000; // jmp

        for (i = 7; i < 16; i = i + 1) begin
            csLUT[i] = 10'b0000000000;
        end
    end

    always@ (Opcode)
    begin
        cntrl_sig = csLUT[Opcode];
    end

    assign Branch = cntrl_sig[9:8];
    assign Jump = cntrl_sig[7];
    assign MemRead = cntrl_sig[6];
    assign MemWrite = cntrl_sig[5];
    assign ALUSrc = cntrl_sig[4];
    assign MemtoReg = cntrl_sig[3];
    assign RegWrite = cntrl_sig[2];
    assign ALUOp = cntrl_sig[1:0];
endmodule
```

Discussion and Simulation

Testbench images:

Adder:

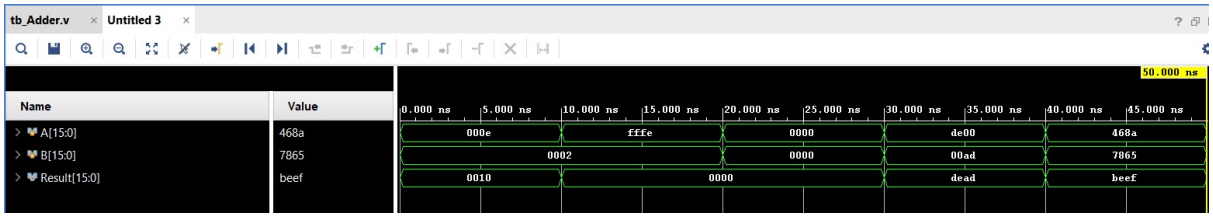
0 to 10ns: Inputs are A = 000E and B = 0002. Sum = is 10 (Basic Addition) (CORRECT)

10 to 20ns: Inputs are A = FFFE and B = 0002. Sum = 0000 (Overflow) (CORRECT)

20 to 30ns: Inputs are A = 0000 and B = 0000. Sum = 0000 (Zero Case) (CORRECT)

30 to 40ns: Inputs are A = DE00 and B = 00AD. Sum = DEAD (Basic Addition) (CORRECT)

40 to 50ns: Inputs are A = 468A and B = 7865. Sum = BEEF (Basic Addition) (CORRECT)



ALU:

0 to 10ns: Inputs are A = 0003, B = 0005, Operation = 0 Output: Result = 0008, Flags = 1 (Add) (CORRECT)

10 to 20ns: Inputs are A = 000A, B = 0003, Operation = 1 Output: Result = 0007, Flags = 1 (Subtract) (CORRECT)

20 to 30ns: Inputs are A = 0004, B = 005A, Operation = 2 Output: Result = 05A0, Flags = 1 (Shift Left Logical) (CORRECT)

30 to 40ns: Inputs are A = 000A, B = 000C, Operation = 3 Output: Result = 0008, Flags = 1 (Bitwise AND) (CORRECT)

40 to 50ns: Inputs are A = 0006, B = 0D20, Operation = 4 Output: Result = 0034, Flags = 1 (Shift Right Logical) (CORRECT)

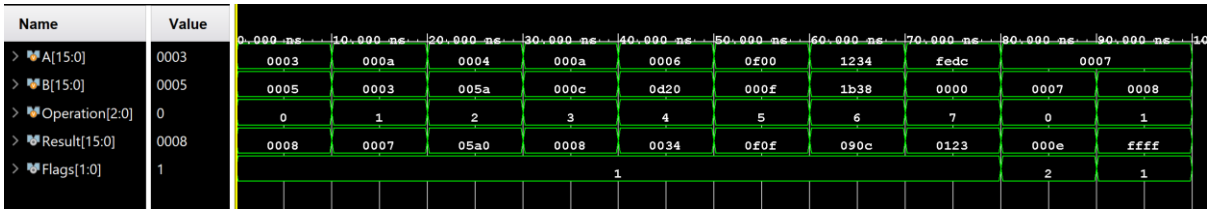
50 to 60ns: Inputs are A = 0F00, B = 000F, Operation = 5 Output: Result = 0F0F, Flags = 1 (Bitwise OR) (CORRECT)

60 to 70ns: Inputs are A = 1234, B = 1B38, Operation = 6 Output: Result = 090C, Flags = 1 (Bitwise XOR) (CORRECT)

70 to 80ns: Inputs are A = FEDC, B = 0000, Operation = 7 Output: Result = 0123, Flags = 1 (Bitwise NOT on A) (CORRECT)

80 to 90ns: Inputs are A = 0007, B = 0007, Operation = 0 Output: Result = 000E, Flags = 2 (Add) (CORRECT)

90 to 100ns: Inputs are A = 0007, B = 0008, Operation = 1 Output: Result = FFFF, Flags = 1 (Subtract / Negative Result) (CORRECT)



ALUControl:

0 to 10ns: Inputs ALUOp = 0 and Funct = x. Operation = 0 (Add) (CORRECT)

10 to 20ns: Inputs ALUOp = 1 and Funct = x. Operation = 1 (Subtract) (CORRECT)

20 to 30ns: Inputs ALUOp = 2 and Funct = 0. Operation = 0 (R-Type Add) (CORRECT)

30 to 40ns: Inputs ALUOp = 2 and Funct = 1. Operation = 1 (R-Type Subtract) (CORRECT)

40 to 50ns: Inputs ALUOp = 2 and Funct = 2. Operation = 2 (R-Type Shift Left Logical) (CORRECT)

50 to 60ns: Inputs ALUOp = 2 and Funct = 3. Operation = 3 (R-Type Bitwise AND) (CORRECT)

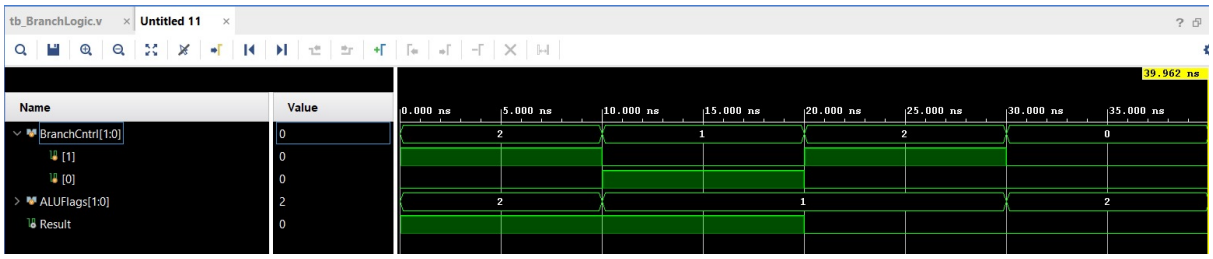
BranchLogic:

0 to 10ns: Inputs are BranchCntrl = 2, ALUFlags = 2 Output: Result = 1 (BEQ Met) (CORRECT)

10 to 20ns: Inputs are BranchCntrl = 1, ALUFlags = 1 Output: Result = 1 (BNE Met) (CORRECT)

20 to 30ns: Inputs are BranchCntrl = 2, ALUFlags = 1 Output: Result = 0 (BEQ Failed) (CORRECT)

30 to 40ns: Inputs are BranchCntrl = 0, ALUFlags = 2 Output: Result = 0 (No Branch Triggered) (CORRECT)



ControlUnit:

0 to 10 ns (R-Type)**:** Input Opcode = 0 Output: RegWrite = 1, ALUOp = 1 (CORRECT)

10 to 20 ns (lw): Input Opcode = 1 Output: MemRead = 1, ALUSrc = 1, MemtoReg = 1, RegWrite = 1 (CORRECT)

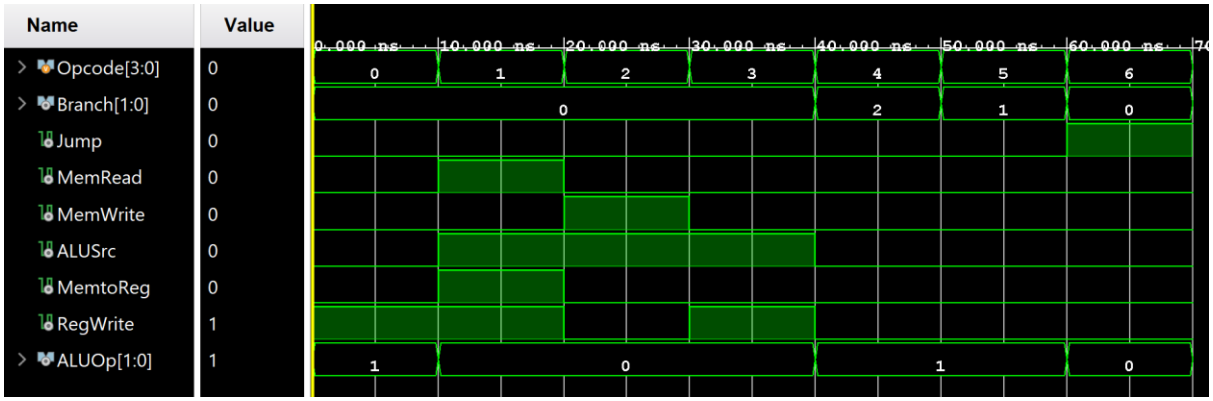
20 to 30 ns (sw): Input Opcode = 2 Output: MemWrite = 1, ALUSrc = 1 (CORRECT)

30 to 40 ns (addi): Input Opcode = 3 Output: ALUSrc = 1, RegWrite = 1 (CORRECT)

40 to 50 ns (beq): Input Opcode = 4 Output: Branch = 2, ALUOp = 1 (CORRECT)

50 to 60 ns (bne): Input Opcode = 5 Output: Branch = 1, ALUOp = 1 (CORRECT)

60 to 70 ns (jmp): Input Opcode = 6 Output: Jump = 1 (CORRECT)



DataMemory:

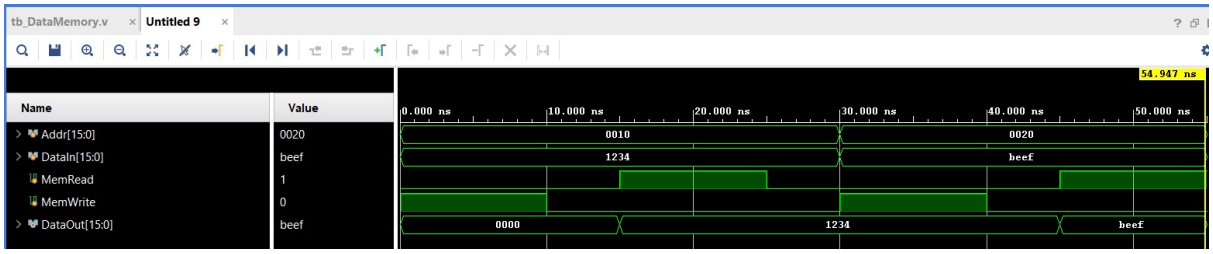
0 to 10 ns (Write): Inputs are Addr = 0010, DataIn = 1234, MemWrite = 1 Output: DataOut = 0000 (Memory Updated) (CORRECT)

10 to 20 ns (Read): Inputs are Addr = 0010, MemRead = 1 Output: DataOut = 1234 (Data Retrieval) (CORRECT)

20 to 30 ns (Idle): Inputs are MemRead = 0, MemWrite = 0 Output: DataOut = 1234 (Latch Previous Value) (CORRECT)

30 to 40 ns (Write): Inputs are Addr = 0020, DataIn = beef, MemWrite = 1 Output: DataOut = 1234 (Memory Updated) (CORRECT)

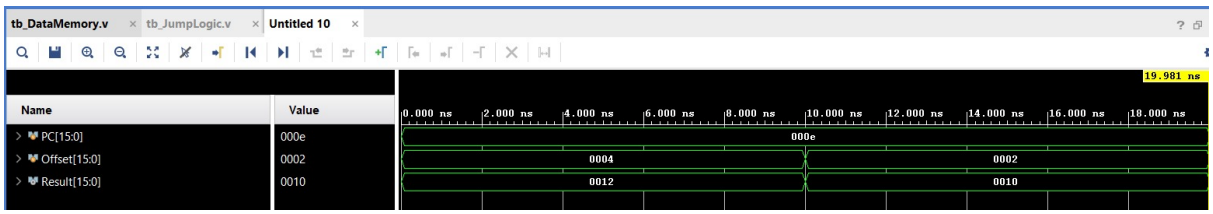
40 to 50 ns (Read): Inputs are Addr = 0020, MemRead = 1 Output: DataOut = beef (Data Retrieval) (CORREC)



JumpLogic:

0 to 10 ns (Jump Calculation): Inputs are PC = 000e, Offset = 0004 Output: Result = 0012 (CORRECT because 0xe + 0x4 = 0x12)

10 to 20 ns (Jump Calculation): Inputs are PC = 000e, Offset = 0002 Output: Result = 0010 (CORRECT because 0xe + 0x2 = 0x10)



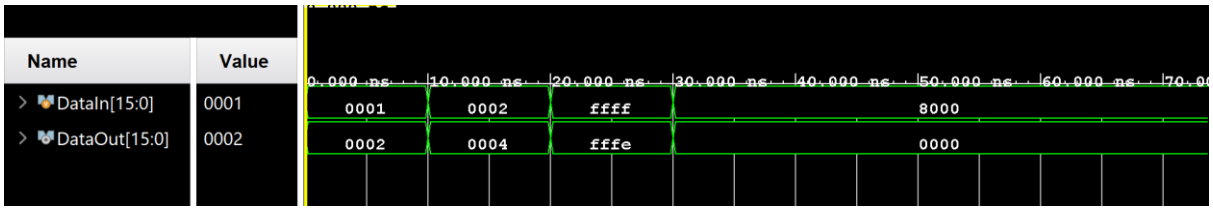
LeftShifter:

0 to 10 ns (Left Shift): Input: DataIn = 0001 Output: DataOut = 0002 (Multiplied by 2) (CORRECT)

10 to 20 ns (Left Shift): Input: DataIn = 0002 Output: DataOut = 0004 (Multiplied by 2) (CORRECT)

20 to 30 ns (Left Shift/Negative): Input: DataIn = ffff Output: DataOut = fffe (CORRECT)

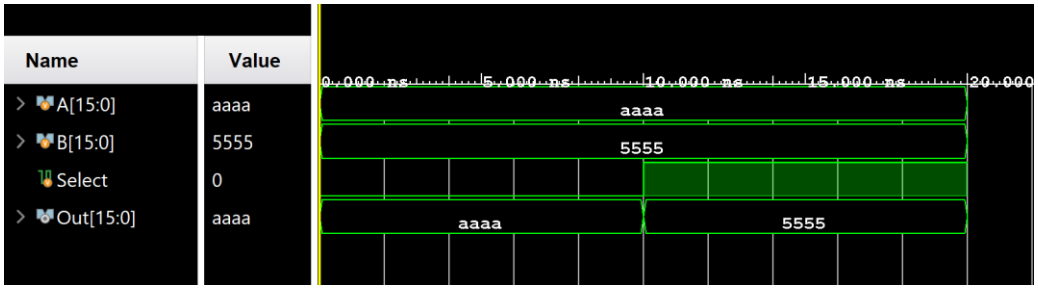
30 to 70 ns (Left Shift/Overflow): Input: DataIn = 8000 Output: DataOut = 0000 (Bit 15 shifted out) (CORRECT)



MUX2To1:

0 to 10 ns (Select A): Inputs: A = aaaa, B = 5555, Select = 0. Output: Out = aaaa (Input A chosen) (CORRECT)

10 to 20 ns (Select B): Inputs: A = aaaa, B = 5555, Select = 1. Output: Out = 5555 (Input B Chosen) (CORRECT)



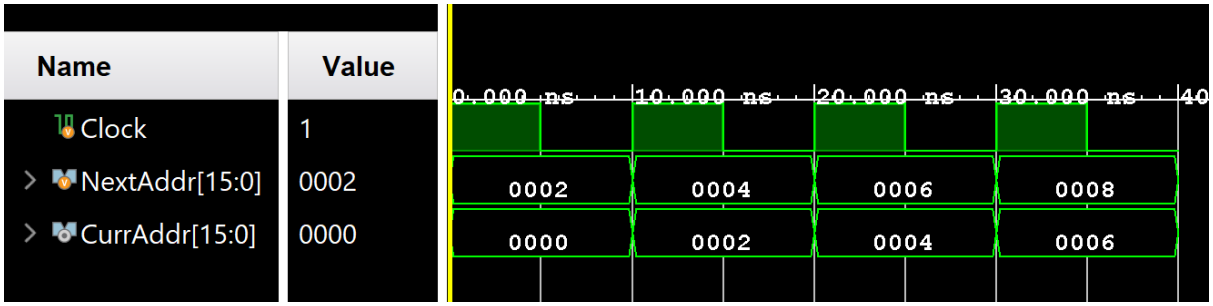
PC:

0 to 10 ns (Update): Input: NextAddr = 0002. Output: CurrAddr = 0000 (Initial State) (CORRECT)

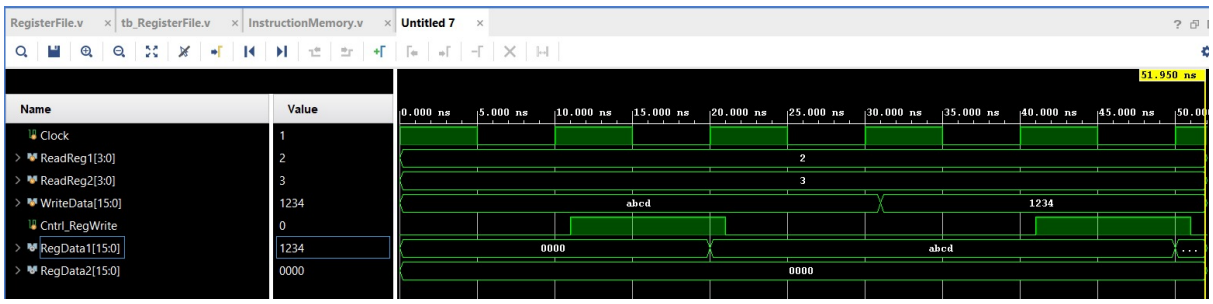
10 to 20 ns (Update): Input: NextAddr = 0004. Output: CurrAddr = 0002 (Address Latched on Clock Edge) (CORRECT)

20 to 30 ns (Update): Input: NextAddr = 0006. Output: CurrAddr = 0004 (Address Latched on Clock Edge) (CORRECT)

30 to 40 ns (Update): Input: NextAddr = 0008. Output: CurrAddr = 0006 (Address Latched on Clock Edge) (CORRECT)



RegFile:



SignExtend:

0 to 10 ns (Positive): Input: ImmediateIn = 3. Output: ImmediateOut = 0003 (Zero Extended) (CORRECT)

10 to 20 ns (Negative): Input: ImmediateIn = e. Output: ImmediateOut = fffe (Sign Bit Preserved) (CORRECT)

20 to 30 ns (Negative): Input: ImmediateIn = 8. Output: ImmediateOut = fff8 (Sign Bit Preserved) (CORRECT)

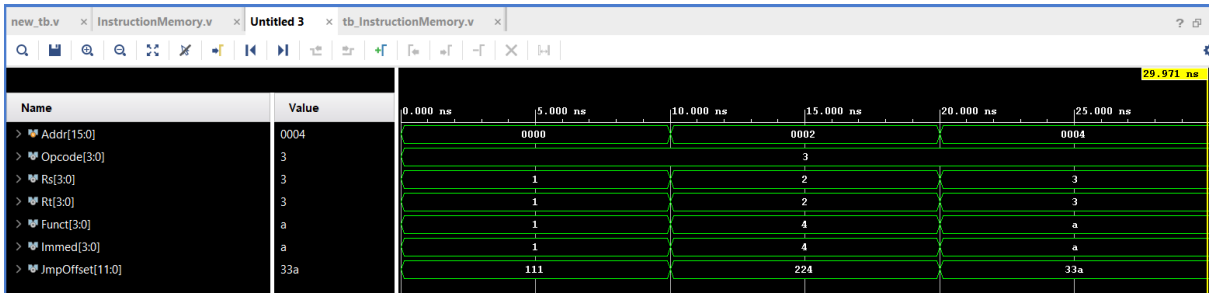
30 to 40 ns (Zero): Input: ImmediateIn = 0. Output: ImmediateOut = 0000 (Zero Extended) (CORRECT)

InstructionMemory:

0 to 10 ns (Fetch Index 0): Input: Addr = 0000. Outputs: Opcode = 3, Rs = 1, Rt = 1, Funct/Immed = 1, JmpOffset = 111 (CORRECT)

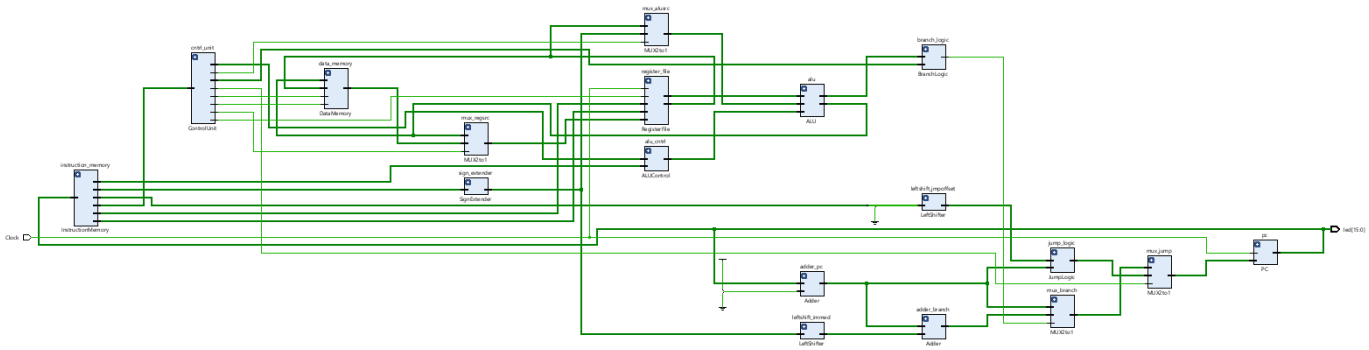
10 to 20 ns (Fetch Index 1): Input: Addr = 0002. Outputs: Opcode = 3, Rs = 2, Rt = 2, Funct/Immed = 4, JmpOffset = 224 (CORRECT)

20 to 30 ns (Fetch Index 2): Input: Addr = 0004. Outputs: Opcode = 3, Rs = 3, Rt = 3, Funct/Immed = a, JmpOffset = 33a (CORRECT)

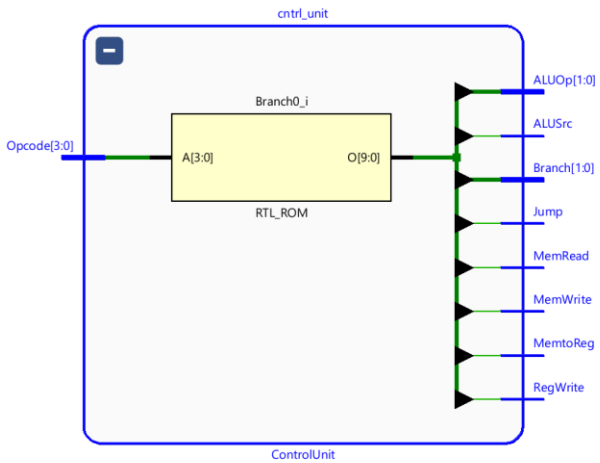


Schematics

Datapath



Control Unit



References

1) *Madhavan, R. (n.d.). Quick Reference for Verilog ® HDL.* *https://web.stanford.edu/class/ee183/handouts_win2003/VerilogQuickRef.pdf*

2) *AMD Technical Information Portal. (2026). Amd.com.* *https://docs.amd.com/r/en-US/ug896-vivado-ip/Vivado-Design-Suite-Documentation*

3) *IEEE Std 1364™-2005 IEEE Standard for Verilog Hardware Description Language IEEE Computer Society Sponsored by the Design Automation Standards Committee. (2006).* *https://www.eg.bucknell.edu/~csci320/2016-fall/wp-content/uploads/2015/08/verilog-std-1364-2005.pdf*

Conclusion

The design and implementation of our single-cycle, 16-bit processor consists of 8 major components: the program counter, control unit, instruction memory, register file, ALU, data memory, a branch logic module, and a jump logic module. Minor components included 2-to-1 multiplexers, 16-bit adders, sign extenders, and logical-left shifters. Each testbench shows the functionality of the individual modules. This project provided insight on the impact of certain architectural design decisions that affect both the hardware layout and the software structure.